

GeoReferencer 機能仕様書

1. プロジェクト概要

1.1 アプリケーションの目的

GeoReferencerは、PNG画像（ハイキングマップなど）を国土地理院の地理院地図上に精密にジオリファレンス（地理的位置合わせ）することを専門とするWebアプリケーションです。最小二乗法による6パラメータアフィン変換技術を用いて、高精度な画像位置合わせを実現し、画像内座標をGPS座標に変換してGeoJSONなどの形式でエクスポートします。

1.2 主要機能

- **GPS座標データの読み込み・表示**（Excel形式、複数ファイル対応／追記読み込み）
- **PNG画像ファイルの読み込み・地図上オーバーレイ表示**
- **JSONファイル連携による画像内座標データの読み込み**（ポイント／ルート／スポット／エリア／複合形式の自動判定）
- **精密アフィン変換によるジオリファレンス**（最小二乗法、Web Mercator経由）
- **エリア（多角形領域）の管理・表示**
- **変換済みGPS座標データのGeoJSONエクスポート**
- **国土地理院APIによる標高データ取得**（ポイント／ルート中間点／スポット／エリア頂点を選択取得）
- **自動ポイントマッチング機能**（IDベース）

1.3 技術的特徴

- **完全ES6モジュール構成**（モジュラーアーキテクチャ、15ファイル）
- **ローカル完結型**（サーバーレス、ファイルベース処理）
- **非同期初期化**（Promise-based確実な初期化）
- **精密座標変換**（Web Mercator ↔ WGS84）
- **標高データ取得**（国土地理院標高API、0.5秒/件のレート制限対応）
- **File System Access API対応**（保存ダイアログ／フォルダ記憶。未対応ブラウザではダウンロードフォルバック）

2. システム構成

2.1 アーキテクチャ概要

GeoReferencerは完全なES6モジュール構成を採用し、機能別に分離されたコアモジュールからなるモジュラーアーキテクチャです。Firebaseなどの外部バックエンド依存を排除し、ローカルファイル操作を中心とした独立性の高い設計となっています。

2.2 モジュール構成と依存関係

GeoReferencerApp (app-main.js)

├─ コア機能

├─ MapCore (map-core.js)

[地図初期化・専用ペイン管理・スケール／ズームコントロール]

```

|   |─ ImageOverlay (image-overlay.js)      [PNG画像オーバーレイ処理]
|   |─ GPSTData (gps-data.js)              [GPS／Excelデータ処理・マージ・地図表示]
|   |─ Georeferencing (georeferencing.js)   [精密アフィン変換処理・ID マッチング]
|   |   └─ AffineTransformation (affine-transformation.js) [アフィン変換係数計算]
|   |─ RouteSpotHandler (route-spot-handler.js) [ルート・スポット・ポイントデータ管理／自動
JSON判定]
|   |─ AreaHandler (area-handler.js)        [エリア(多角形)管理／頂点標高管理]
|   |─ CoordinateDisplay (coordinate-display.js) [画像内座標の表示・マーカー管理]
|   |─ ElevationFetcher (elevation-fetcher.js) [国土地理院標高API連携]
|   |   └─ DataImporter (data-importer.js)    [GPS／PNG／JSONの読み込み統合]
└─ ユーティリティ
    |─ UIHandlers (ui-handlers.js)          [UI操作ハンドラー・件数表示]
    |─ FileHandler (file-handler.js)        [ファイル保存／読み込み・Excel検証変換]
    |─ Utils (utils.js)                    [Logger、エラーハンドリング]
    |─ MathUtils (math-utils.js)           [Web Mercator変換・行列演算・カスタムマーカ
-]
    └─ Constants (constants.js)            [CONFIG／DEFAULTS／CSS_CLASSES／
LOG_LEVELS]

```

2.3 外部依存関係

- **Leaflet.js v1.9.4:** 地図レンダリング (CDN経由)
- **SheetJS v0.18.5:** Excelファイル処理 (CDN経由)
- **国土地理院タイル:** <https://cyberjapandata.gsi.go.jp/xyz/std/{z}/{x}/{y}.png> (minZoom 2、maxZoom 18)
- **国土地理院標高API:**
<https://cyberjapandata2.gsi.go.jp/general/dem/scripts/getelevation.php>

2.4 ファイル構成

```

GeoReferencer/
├─ index.html          # メインHTML (操作パネル定義含む)
├─ styles.css          # 統合CSS
├─ README.md          # プロジェクト概要
├─ docs/              # ドキュメント
|   └─ funcspec-202604.md # 機能仕様書 (本書)
|   └─ UsersGuide-202604.md # 利用者の手引
|   └─ dataspec-json-202604.md # 入力JSON仕様
|   └─ dataspec-geojson-202604.md # 出力GeoJSON仕様
└─ js/                # JavaScriptモジュール
    └─ app-main.js      # メインアプリケーション
    └─ map-core.js      # 地図コア・ペイン管理
    └─ image-overlay.js  # 画像オーバーレイ処理
    └─ gps-data.js       # GPS／Excelデータ処理
    └─ georeferencing.js # 精密アフィン変換処理
    └─ affine-transformation.js # アフィン係数計算
    └─ route-spot-handler.js # ルート・スポット・ポイント管理
    └─ area-handler.js   # エリア (多角形) 管理
    └─ coordinate-display.js # 画像座標表示・マーカー管理
    └─ elevation-fetcher.js # 標高データ取得

```

— data-importer.js	# データ読み込み統合
— ui-handlers.js	# UI操作ハンドラー
— file-handler.js	# ファイル処理統合（保存・Excel変換）
— math-utils.js	# 数学・座標変換・マーカー
— utils.js	# ロガー・エラーハンドラ
— constants.js	# 定数定義

3. 機能詳細

3.1 メインアプリケーション (GeoReferencerApp)

責任範囲: アプリケーション全体の初期化・統合・イベント管理

主要メソッド:

- `init()`: 非同期アプリケーション初期化（地図 → モジュール → ハンドラー）
- `initializeModules()`: 各モジュールの依存関係を考慮した初期化
- `setupEventHandlers()`: 操作パネルのUIイベント設定
- `handleMatchPoints()`: ジオリファレンス実行の統合処理
- `handleExportGeoJson()`: GPS変換済みデータのGeoJSON出力
- `handleFetchElevation()`: 標高データ取得処理
- `collectGeoreferencedData()`: ジオリファレンス済みデータをFeatureCollectionへ収集
- `getGeoJsonFileName()`: 出力ファイル名の自動生成
- `roundCoordinate()`: 座標の小数点5桁丸め
- `getElevationStats()`: 標高未取得件数集計
- `updateElevationCounts()`: 標高未取得件数のUI反映

データフロー:

PNG読み込み → GPS Excel読み込み → JSON読み込み（任意） →
ジオリファレンス実行 → GPS変換 → 標高取得（任意） → GeoJSON出力

3.2 データ読み込み機能 (DataImporter)

責任範囲: 各種ファイル（Excel、PNG、JSON）の読み込み処理

主要機能:

- **GPS Excel読み込み** (`handleGpsExcelLoad`): 複数ファイル対応。既存データがあれば「追記／中止」を確認、座標一致でスキップ。最大行数は `CONFIG.MAX_EXCEL_ROWS` (1000) に制限。
- **PNG画像読み込み** (`handlePngLoad`): 既存データがあれば確認後にクリア。ファイル名（拡張子なし）を `currentPngFileName` として保持。
- **ポイント(座標)JSON読み込み** (`handlePointCoordJsonLoad`): 単一の `points` JSONを読み込み、画像上にマーカー表示。
- **ルート・スポット(座標)JSON読み込み** (`handleRouteSpotJsonLoad`): 複数ファイル対応。
`detectJsonType()` で自動判定。

- **汎用JSON読み込み** (`handleJsonLoad`): GeoJSON / 独自JSON / 複合形式の混在を自動判定し、追記読み込みに対応（重複は座標・IDで判定しスキップ）。

3.3 地図コア機能 (MapCore)

責任範囲: Leaflet地図の初期化・専用ペイン管理・スケールコントロール

主要機能:

- **非同期初期化:** Promise-baseの確実な地図初期化 (`initPromise`)
- **コントロール配置:**
 - スケールバー: 右下、メートル単位、最大幅150px
 - ズームコントロール: 左上 + 右下（2か所）
- **ペイン管理 (z-index) :**

ペイン	z-index	用途
<code>routeLines</code>	600	経路ライン
<code>gpsMarkers</code>	610	GPSポイント（緑）
<code>pointJsonMarkers</code>	620	画像内ポイント（赤）
<code>wayPointMarkers</code>	630	ルート中間点（オレンジ/ダイヤ）
<code>spotMarkers</code>	630	スポット（青）

- **初期表示:** 中心 = 箕面大滝(34.853667, 135.472041)、ズーム 15

3.4 画像オーバーレイ処理 (ImageOverlay)

責任範囲: PNG画像の読み込み・表示・境界計算・アフィン変換対応

主要機能:

- **画像読み込み:** PNG専用FileReader処理
- **境界計算:** Mercator投影補正を考慮した精密境界計算
- **アフィン変換対応:** 変換行列に基づく表示位置更新
- **コールバック:** 画像更新時に登録ハンドラ（マーカー再配置等）を呼び出し

3.5 GPS/Excelデータ処理 (GPSPData)

責任範囲: Excelファイル由来のGPSポイントの管理・地図表示

対応フォーマット (Excelファイル) :

列名	必須	説明
ポイントID	○	ポイント固有ID
名称	○	地点名
緯度	○	10進度 (-90~90)
経度	○	10進度 (-180~180)

列名	必須	説明
標高	-	メートル
備考	-	任意

機能:

- 検証: 列存在確認・座標範囲チェック・数値形式検証
- マージ: `mergePoints()` で座標一致による重複スキップ
- 表示: 緑色円形マーカーで地図表示

3.6 精密アフィン変換処理 (Georeferencing)

責任範囲: 最小二乗法による6パラメータアフィン変換・精度計算・座標同期

技術仕様:

- **変換方式:** 最小二乗法による6パラメータアフィン変換 (Web Mercator経由)
- **変換係数:** `a`, `b`, `c`, `d`, `e`, `f`
 - `webMercatorX = a*imageX + b*imageY + c`
 - `webMercatorY = d*imageX + e*imageY + f`
- **最小制御点数:** 3点以上 (推奨: 4点以上)
- **マッチング:** 画像JSONの`id`とGPS Excelの`ポイントID`を完全一致で対応付け
- **画像更新コールバック:** 変換後の画像位置変更時にポイント/ルート/スポット/エリアを同期

3.7 標高データ取得 (ElevationFetcher)

責任範囲: 国土地理院APIから標高データ取得・各データへの設定

主要機能:

- **標高取得** (`fetchElevation(lng, lat)`): 国土地理院標高APIを呼び出し、標高 (小数点1桁丸め) を返却
- **対象別一括取得:**
 - `fetchAndSetPointsElevation()`: ポイント (画像座標→GPS変換後)
 - `fetchAndSetRouteMarkersElevation()`: ルートマーカー (中間点含む)
 - `fetchAndSetSpotMarkersElevation()`: スポットマーカー
 - `fetchAndSetAreaVerticesElevation()`: エリア頂点 (`AreaHandler.getAllVertices()` / `setVertexElevation()`連携)
- **既設定スキップ:** 取得済みは再取得しない
- **レート制限:** 0.5秒/件 (`DELAY_MS = 500`)
- **進捗コールバック:** 件数進捗をUIへ通知

3.8 ルート・スポットデータ管理 (RouteSpotHandler)

責任範囲: 画像座標データ (JSON) の管理・表示・自動判定

自動判定 (`detectJsonType`): 入力JSONを以下に分類

- `combined: data.{points/routes/spots/areas}` ラッパー形式

- `route`: `routeInfo.startPoint/endpoint + points[type="waypoint"]`
- `spot`: `spots[]` または `name` を持つ単独オブジェクト
- `point`: `points[]` で `type !== "waypoint"`、`id` または `name` を持つもの
- `area`: `areas[].vertices[]` を持つもの

主要機能:

- **データ保持**: `pointData`、`routeData`、`spotData`、`pointMarkers`、`routeMarkers`、`spotMarkers`
- **マージ**: ID/座標の双方向比較による重複検出 (`isSameRoute`、`isSameSpot`)
- **マーカー表示**: ルート中間点 (ダイヤ)、スポット (青)、ポイント (赤)
- **メタ情報**: 各マーカーの `__meta` に `origin` (`image/firebase/json/gps`)、`imageX/Y`、`routeId/spotId`、`elevation` 等を保持

3.9 エリア管理 (AreaHandler)

責任範囲: エリア (多角形領域) の取り込み・地図表示・頂点標高管理

主要機能:

- **取り込み** (`importAreas`): 画像座標 `vertices[]` を保持。`id` 未指定時は `area_{index}` を自動付与。名称は `areaName/name/description` の順でフォールバック、いずれも空なら `エリア {index+1}`。
- **表示**: ピンク多角形 (HotPink境界 / LightPink塗り 30%) + 頂点ピンク菱形マーカー (6px)
- **同期** (`syncAreaPositions`): アフィン変換確定時にポリゴン・頂点マーカー位置を再計算
- **頂点取得・更新**:
 - `getAllVertices()`: `[{areaId, areaName, vertexIndex, lat, lng, elevation}]` を返却
 - `setVertexElevation(areaId, index, elevation, areaName?)`: 一意キー `areaId` で標高を反映 (信頼源は `areas`、表示用 `areaPolygons` にも反映)
- **頂点数集計** (`getVertexCount`): `polygon.__meta.vertices` の合計

3.10 座標表示 (CoordinateDisplay)

責任範囲: 画像座標→GPS変換でのマーカー作成・ポップアップ生成

主要機能:

- **画像座標抽出** (`extractImageCoordinates`): 配列 / `points` / `routes.points/waypoints` / `spots/data` ラッパー の各形式を解釈し、座標フィールドは `imageX/imageY` を優先、無ければ `x/y` も受け付け
- **マーカー種別決定** (`determineMarkerType`): `waypoint`→`wayPoint`、`name付きspot`→`spot`、その他→`pointJSON`
- **ポップアップ**:
 - `createGpsPopupContent`: GPSポイント (`pointId`
表示名)
 - `createRouteWaypointPopupContent`: 中間点 (中間点-{N})
 - `createSpotPopupContent`: スポット名

3.11 ファイル処理統合 (FileHandler)

責任範囲: ファイル読み込み・保存・Excel検証変換

機能:

- **保存** (`saveDataWithUserChoice`):
 - File System Access API利用可: `showSaveFilePicker` でダイアログ表示。前回ファイルの親フォルダから開始可能。拡張子 `.geojson` を自動付与
 - 未対応／失敗時: Blob + `<a download>` のダウンロードフォールバック
- **読み込み**: `loadJsonFile` / `loadExcelFile` / `loadImageFile`
- **検証** (`isExcelFile`/`isPngFile`/`isJsonFile`): 拡張子・MIMEタイプチェック
- **Excel検証変換** (`validateAndConvertExcelData`): 必須／オプション列検出、緯度経度の数値・範囲検証

3.12 UI操作ハンドラー (UIHandlers)

責任範囲: 件数表示・マッチング結果反映・UI状態管理

機能:

- 件数表示: GPSポイント、ポイント／ルート／スポット／エリア
- マッチング結果: 一致数、不一致ID表示
- 全件数クリア (PNG再読み込み時)

3.13 ロガー／エラーハンドラ (Utils)

- **Logger**: モジュール名タグ付きINFO/WARN/ERROR出力 (debug用 `?debug` クエリ対応)
- `errorHandler.handle(error, userMsg, context)`: エラー文言の整形＋ユーザー通知

3.14 数学・座標変換 (MathUtils)

- Web Mercator ↔ WGS84変換 (`lonToWebMercatorX` 等)
- メートル/ピクセル算出 (`calculateMetersPerPixel`)
- アフィン変換適用 (`applyAffineTransform`)
- 行列演算: 転置／積／ガウス・ジョーダン法
- カスタムマーカー生成 (`createCustomMarker`)

4. ユーザーインターフェース

4.1 UI構成

- **地図エリア**: 画面全体に表示される国土地理院地図
- **メッセージエリア**: 画面上部の一時メッセージ表示 (成功・エラー・進捗、3秒間)
- **制御パネル** (右上の操作パネル):
 - **読み込みセクション**:
 - ラジオボタン: ポイントGPS／PNG画像／JSONファイル
 - 「読み込み」ボタン
 - 件数表示: GPSポイント、ポイント／ルート／スポット／エリア
 - PNGファイル名表示
 - **マッチングセクション**:
 - 「画像の重ね合わせ (ジオリファレンス)」ボタン
 - 一致ポイント数、不一致ポイント表示
 - **標高セクション**:
 - 「標高取得」ボタン (ジオリファレンス後に有効化)

- チェックボックス: ポイント／ルート中間点／スポット／エリア頂点
- 各対象の標高未取得件数表示
- 進捗バー（取得中のみ表示）
- **保存ボタン**: 「変換後のGPS値をGeoJSONファイルに保存」（ジオリファレンス後に有効化）
- 注記: 「（ポイントGPSは、GeoJSONファイルへの格納対象外）」

4.2 ファイル読み込み機能

ポイントGPS読み込み（複数ファイル対応）

- **形式**: Excel (.xlsx)
- **動作**: 既存データがある場合「追記／中止」確認。追記時は座標一致でスキップ
- **フィードバック**: 件数表示と完了メッセージ

PNG画像読み込み

- **形式**: PNG
- **動作**: 既存画像がある場合は確認後にクリア。ファイル名を保持し、JSON読み込みやファイル名生成に利用
- **フィードバック**: ファイル名を表示

JSONファイル読み込み（複数ファイル対応）

- **形式**: JSON（独自スキーマ／GeoJSON／複合形式の自動判定）
- **動作**: 既存データがある場合「追記／中止」確認。追記時は重複スキップ
- **前提条件**: PNG画像が読み込まれていない場合は読み込み拒否（警告表示）
- **フィードバック**: ポイント・ルート・スポット・エリアの各件数を表示

4.3 ジオリファレンス操作

- **実行**: 「画像の重ね合わせ（ジオリファレンス）」ボタン
- **前提**: PNG画像 + (GPSデータ or ポイントJSON)
- **処理**: ID完全一致でマッチング → 6パラメータアフィン変換係数算出 → 画像位置／全マーカー更新
- **結果表示**: 一致ポイント数、不一致ポイントID、メッセージ表示
- **副作用**: GeoJSON保存ボタンと標高取得ボタンが有効化される

4.4 標高取得

- **対象選択**: 4つのチェックボックス（ポイント／ルート中間点／スポット／エリア頂点）
- **進捗**: バー + 「{対象}の標高を取得中: {current} / {total}」テキスト
- **完了**: 取得成功・失敗件数を表示。3秒後にバー非表示
- **複合JSON対応**: `imageCoordinateMarkers` 由来のwaypoint／spot／pointJSONも、対応する `routeSpotHandler` データが空の場合に取得対象とする

4.5 データ出力

- **GeoJSON保存**: ジオリファレンス済み全データを `FeatureCollection` で出力
- **保存ダイアログ**: File System Access API対応時は `showSaveFilePicker`、未対応時はダウンロード
- **ファイル名規則**: {略称}-GPS-P{N}_R{N}_S{N}_A{N}-YYYYMMDD.geojson

- 略称 = PNGファイル名先頭から区切り文字（-、_、 、.）の前まで
- 件数0の項目は省略、すべて0なら件数部省略
- 例: minoo-GPS-P5_R2_S3-20260427.geojson

5. データ構造

5.1 入力JSONフォーマット（自動判定）

詳細は [dataspec-json-202604.md](#) 参照。下記5形式を自動判定。

形式	主要フィールド	座標フィールド
Point	<code>points[].{id,name,imageX,imageY}</code>	<code>imageX/imageY</code>
Route	<code>routeInfo.{startPoint,endPoint} + points[type="waypoint"].{imageX,imageY}</code>	<code>imageX/imageY</code>
Spot	<code>spots[].{name,imageX,imageY}</code>	<code>imageX/imageY</code>
Area	<code>areas[].vertices[].{x,y}</code>	<code>x/y</code>
Combined	<code>data.{points,routes,spots,areas}</code>	<code>x/y または imageX/imageY</code>

5.2 GeoJSON出力フォーマット

詳細は [dataspec-geojson-202604.md](#) 参照。

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "properties": {
        "id": "A-01",
        "name": "登山口",
        "type": "point",
        "source": "image_transformed",
        "description": "画像ポイント（GPS変換済）"
      },
      "geometry": {
        "type": "Point",
        "coordinates": [135.47204, 34.85367, 150.5]
      }
    }
  ]
}
```

Feature種別:

type	Geometry	主要properties
------	----------	--------------

type	Geometry	主要properties
point	Point	id, name, source, description
spot	Point	id (spotNN_name), name, source, description
route	LineString	id (route_start_to_end), name, startPoint, endPoint, startPointGPS, endPointGPS, source, description
area	Polygon	id, name, source, description（自動閉合）

座標は [lng, lat] または [lng, lat, elevation]、すべて小数点5桁丸め。

6. 制限事項

- **画像形式:** PNGのみ対応
- **ブラウザ:** ES6モジュール対応必須。File System Access API非対応ブラウザではダウンロード保存にフオールバック
- **API制限:** 国土地理院標高APIのアクセス制限準拠（0.5秒/件）
- **座標系:** WGS84（EPSG:4326）のみ
- **Excel行数:** 1000行（ヘッダー含む、CONFIG.MAX_EXCEL_ROWS）
- **アフィン変換:** 最低3点（推奨4点以上）の制御点が必要
- **GeoJSON出力対象外:** ポイントGPS（Excel由来）はそのままでは出力しない（画像由来データのGPS変換結果のみ出力）

7. 改訂履歴

- **v2.1** (2026-04-27): エリア（多角形）取り扱いの仕様明文化、エリア頂点の標高取得追記。ファイル名規則の集計ルール反映。複合JSON形式（data ラッパー）の判定／インポートを正式化。dataspec-json-202604.md / dataspec-geojson-202604.md への参照を追加。
- **v2.0** (2026-02-13): Firebase依存を完全に排除し、ローカルファイルベースのフローに刷新。GeoJSON エクスポート機能を追加。
- **v1.0** (2025-12-01): 初版リリース（Firebase版）

作成日: 2026年4月27日 バージョン: 2.1